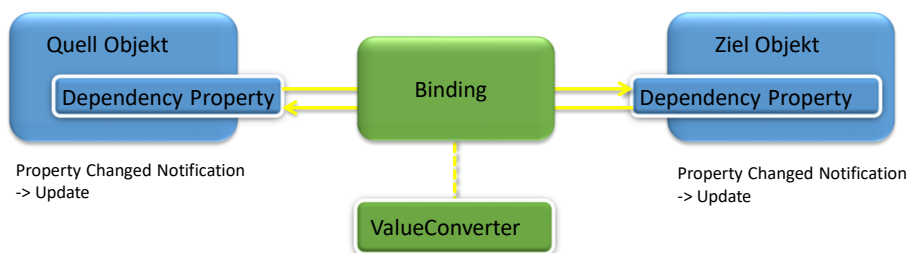


Binding

Property Binding

- Property Binding ermöglicht Datenzugriff/Datenänderung zwischen zwei UI-Controls
- Bindungen müssen immer im Target-Objekt definiert werden
- Aktualisierungen erfolgen automatisch
- Mode vom Binding gibt die Richtung(en) an
- Converter können den Wert/Datentyp manipulieren



Das PropertyBinding in WPF ermöglicht die Bindung von Eigenschaften zwischen zwei UI-Elementen. Dabei gibt es eine Quelle (Source) und ein Ziel (Target), zwischen denen die Bindung hergestellt wird. Die Definition der Bindung erfolgt immer in der Target-Eigenschaft.

Quelle und Ziel sind jeweils Eigenschaften von UI-Elementen (Controls und Container). Durch die Bindung wird die Quelleneigenschaft automatisch mit der Zielschicht synchronisiert, so dass Änderungen an der Quelle automatisch im Ziel reflektiert werden. Sind die Datentypen unterschiedlich muss in der Regel eine Converterklasse die Umwandlung erledigen.

Die Aktualisierung erfolgt in beide Richtungen: Wenn sich der Wert der Quelleneigenschaft ändert, wird der Wert automatisch an das Ziel übertragen. Gleichzeitig kann auch das Ziel den Wert ändern und er wird an die Quelle zurückgebunden. Dies ermöglicht eine bidirektionale Kommunikation zwischen den UI-Elementen.

Die PropertyBinding in WPF basiert auf DependencyPropertyes. DependencyPropertyes sind spezielle Eigenschaften in WPF, die zusätzliche Funktionen bieten. Anders als in ‚normalen‘ Eigenschaften werden hier die Werte nicht in den Objekten, sondern in einem globalen Dictionary (ValueStore) gespeichert. Dadurch verfügen DependencyPropertyes zum einen immer über einen Standard-Wert, zum anderen können sie sich bei Veränderungen automatisch gegenseitig über diese informieren.

Binding-Mode

- OneTime: Target wird einmalig mit Wert der Source initialisiert
- OneWay: Target wird bei Änderung der Source aktualisiert
- OneWayToSource: Source Property wird bei Änderung des Target aktualisiert
- TwoWay: Target wird bei Änderung der Source aktualisiert und umgekehrt

Die Mode-Eigenschaft des Binding-Objekts definiert, wie die Datenbindung zwischen der Quelle (Source) und dem Ziel (Target) synchronisiert wird. Sie beeinflusst das Verhalten der Datenaktualisierung und ermöglicht die Steuerung des Datenflusses.

Es gibt verschiedene Optionen für die Mode-Eigenschaft:

1. OneWay: Hier wird die Datenbindung in eine Richtung festgelegt, d.h. Änderungen an der Quelle werden automatisch im Ziel reflektiert. Wenn sich jedoch der Wert im Ziel ändert, wird er nicht an die Quelle zurückgebunden.
2. TwoWay: In diesem Modus wird eine bidirektionale Datenbindung hergestellt. Änderungen an der Quelle werden automatisch im Ziel aktualisiert, und Änderungen im Ziel werden auch an die Quelle zurückgebunden. Dadurch kann der Wert sowohl von der Quelle als auch vom Ziel geändert werden.
3. OneWayToSource: Hier erfolgt die Datenbindung nur in eine Richtung - vom Ziel zur Quelle. Änderungen im Ziel werden an die Quelle zurückgebunden, aber Änderungen an der Quelle haben keinen Einfluss auf das Ziel. Dies ist hilfreich, wenn man sicherstellen möchte, dass der Wert nur von einer bestimmten Quelle geändert werden kann.
4. OneTime: Mit diesem Modus wird die Datenbindung einmalig hergestellt und danach nicht mehr aktualisiert. Änderungen an der Quelle werden nicht im Ziel reflektiert und umgekehrt. Dies kann nützlich sein, wenn der Wert statisch ist und sich während der Laufzeit nicht ändert.

UpdateSourceTrigger

- Der UpdateSourceTrigger definiert den Zeitpunkt, zum dem die Source aktualisiert wird
- Arten
 - Default: PropertyChanged, Texteigenschaft -> LostFocus
 - Explicit: Aktualisiert bei expliziten Aufruf UpdateSource
 - LostFocus: Aktualisiert wenn Element Fokus verliert
 - PropertyChanged: Update wenn Property Value geändert

Die Eigenschaft UpdateSourceTrigger wird verwendet, um das Verhalten der Datenaktualisierung zwischen dem Ziel (Target) und der Quelle (Source) in einer Datenbindung zu steuern. Sie definiert, wann und unter welchen Bedingungen der Wert im Ziel an die Quelle zurückgebunden wird.

Es gibt verschiedene Optionen für die UpdateSourceTrigger-Eigenschaft:

1. PropertyChanged: Dies ist der Standardwert. Bei jeder Änderung des Werts im Ziel wird die Aktualisierung an die Quelle sofort ausgelöst. Das heißt, der Wert wird automatisch zurückgebunden, sobald er geändert wird.
2. LostFocus: Die Aktualisierung der Quelle erfolgt erst, wenn das Ziel den Fokus verliert, z.B. wenn der Benutzer das Textfeld verlässt oder eine andere Aktion ausführt. Dies ist nützlich, um die Leistung zu verbessern und unnötige Aktualisierungen während der Eingabe zu vermeiden.
3. Explicit: Hier erfolgt die Aktualisierung der Quelle nur auf explizite Anforderung, z.B. durch den Aufruf der Methode UpdateSource(). Dies ermöglicht eine manuelle Kontrolle über den Zeitpunkt der Aktualisierung.
4. Default: Diese Option wird normalerweise nicht explizit festgelegt. Sie verhält sich je nach Kontext unterschiedlich. In den meisten Fällen entspricht sie dem Verhalten von PropertyChanged, kann aber je nach Eigenschaftstyp oder Kontext andere Auswirkungen haben.

Weitere wichtige Eigenschaften

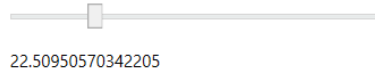
- **FallbackValue**
 - Verwendeter Wert im Target bei fehlgeschlagener Bindung (falscher Datentyp, fehlgeschlagene Initialisierung, etc)
- **TargetNullValue**
 - Verwendeter Wert im Target, wenn die Source den Wert ‚Null‘ ausgibt
- **Delay**
 - Verzögerung in Millisekunden
- **IsAsync**
 - Bindung erfolgt asynchron in separatem Thread -> lange Operationen frieren die UI nicht ein
- **StringFormat**
 - Formartierung der angezeigten Strings im Target

1. **FallbackValue:** Diese Eigenschaft ermöglicht das Festlegen eines Standardwerts, der verwendet wird, wenn das Binding-Objekt nicht auf einen gültigen Wert zugreifen kann. Wenn beispielsweise die Datenquelle nicht verfügbar ist oder das Binding fehlschlägt, wird der FallbackValue verwendet, um das Zielobjekt zu aktualisieren.
2. **TargetNullValue:** Diese Eigenschaft ermöglicht das Festlegen eines spezifischen Werts, der verwendet wird, wenn der Wert der Datenquelle null ist. Wenn beispielsweise der Wert einer Eigenschaft in der Datenquelle null ist, wird der TargetNullValue verwendet, um das Zielobjekt entsprechend zu aktualisieren.
3. **Delay:** Diese Eigenschaft ermöglicht das Festlegen einer Verzögerung (in Millisekunden), bevor das Binding-Objekt aktualisiert wird. Dies ist besonders nützlich, um eine unnötige Aktualisierung der Benutzeroberfläche zu vermeiden, wenn sich der Wert häufig ändert. Durch das Festlegen einer Verzögerung kann das Binding-Objekt warten, bis der Wert stabil ist, bevor es aktualisiert wird.
4. **IsAsync:** Diese Eigenschaft ermöglicht das Asynchronisieren des Datenbindings. Das Datenbinding erfolgt asynchron, was bedeutet, dass der Datenzugriff in einem separaten Thread stattfindet. Dies kann nützlich sein, um die Benutzeroberfläche reaktionsfähig zu halten, wenn der Datenzugriff viel Zeit in Anspruch nimmt.
5. **StringFormat:** Diese Eigenschaft ermöglicht die Formatierung von gebundenen Werten als Zeichenkette. Sie verwendet Platzhalter und Formatierungssymbole, um den Wert entsprechend zu formatieren, z. B. für Währungen oder Datumsangaben.

Beispiel Xaml VS. C#

C#:

```
Binding b = new Binding("Value");  
b.Source = MySlider;  
BindingOperations.SetBinding(MyLabel, Label.ContentProperty, b);
```



Xaml:

```
<Slider Minimum="0" Maximum="100" Width="100" Height="20" x:Name="MySlider"/>  
  
<Label x:Name="MyLabel">  
  <Label.Content>  
    <Binding ElementName="MySlider" Path="Value"/>  
  </Label.Content>  
</Label>
```

Um eine Bindung zu etablieren muss zunächst bestimmt werden, welche Eigenschaft das Target und welche die Source ist. Hiervon hängen der Einsatz bestimmter Funktionen, wie der UpdateSourceTrigger oder der TargetNullValue, ab.

In XAML lagert man dann die Target-Eigenschaft aus und legt dort ein Element vom Typ Binding ein. Hier definiert man über ElementName den x:Name des Source-Objekts und unter Path die Source-Eigenschaft. Zusätzlich können hier nun Eigenschaften wie Mode, UpdateSourceTrigger, FallbackValue, etc gesetzt werden um die Bindung zu spezifizieren.

Innerhalb von C# kann alternativ ein Binding-Objekt unter Angabe von Source und Path erstellt werden und über Die BindingOperations-Klasse der Target-Eigenschaft zugeordnet werden.

Binding

- Markup-Extension { } für verkürzten Zugriff und zum Referenzieren bestehender Objekte

```
<Slider Minimum="0" Maximum="100" Width="100" Height="20" x:Name="MySlider"/>

<!--<Label x:Name="MyLabel">
    <Label.Content>
        <Binding ElementName="MySlider" Path="Value"/>
    </Label.Content>
</Label>-->

<Label x:Name="MyLabel" Content="{Binding ElementName=MySlider, Path=Value}"/>
```

Als verkürzte Schreibweise können Bindung innerhalb von XAML auch in Form einer sogenannten MarkupExtension geschrieben werden. Dies ermöglicht es, auf das Auslagern der Eigenschaft zu verzichten und die Bindung in der Attribut-Syntax zu schreiben.

Dazu wird das Binding-Objekt in das Attribut der Target-Eigenschaft innerhalb von {}-Klammern gesetzt. Innerhalb der Klammern findet eine veränderte Syntax Anwendung. Hier werden Attribute durch Kommas voneinander getrennt und deren Werte nicht durch doppelte Anführungszeichen hervorgehoben (Hierfür können bei Bedarf die einfachen Anführungszeichen verwendet werden).

Auch abseits von Bindings werden MarkupExtensions verwendet um komplexe Objekte in der Attribut-Syntax dazustellen. Die funktioniert aber nicht mit jedem beliebigen Objekt, sondern nur mit solchen, welche von der MarkupExtension-Klasse erben und somit hierfür spezifiziert sind.

Value Converter

- Wenn Typ von Quell und Zielwert nicht übereinstimmen -> zwischenschalten von Convertern
- Vordefinierter Converter WPF -> BooleanToVisibilityConverter
 - Wenn Property Quelle True -> Zielwert Visibility.Visible
 - Wenn Property Quelle False -> Zielwert Visibility.Collapsed
- Implementieren des Interface IValueConverter

Ein ValueConverter ermöglicht innerhalb einer Bindung die Umwandlung eines Werts in einen anderen, um ihn an eine Zielkomponente anzupassen. Ein gängiger Anwendungsfall ist die Konvertierung eines Wertes in ein anderes Datenformat oder eine andere Darstellung bzw. die Manipulation eines Werts, beispielsweise mittels eines Multiplikators. Dazu wird der Converter in die Converter-Eigenschaft des Binding-Objekts eingesetzt.

WPF bietet einige wenige vordefinierte Converter wie z.B. den BooleanToVisibilityConverter. Dieser Converter wird verwendet, um einen booleschen Wert (true oder false) in die Sichtbarkeits-Eigenschaft (Visibility) eines UI-Elements umzuwandeln.

In den meisten Fällen wird eine individuelle Anpassung des Werts benötigt. Hierfür müssen eigene Converter-Klassen geschrieben werden, welche das IValueConverter-Interface implementieren müssen.

IValueConverter

```
public class DoubleToBrushConverter : IValueConverter
{
    //Source->Target:
    public object Convert(object value, Type targetType, object parameter, CultureInfo culture)
    {
        //Rückgabe eines Objekts vom Targetdatentyp, basiert auf value (<- Sourcewert)
    }

    //Target->Source
    public object ConvertBack(object value, Type targetType, object parameter, CultureInfo culture)
    {
        //Rückgabe eines Objekts vom Sourcedatentyp, basiert auf value (<- Targetwert)
    }
}

<Slider x:Name="Sdr_Rot"/>
<Rectangle Width="100" Height="100">
    <Rectangle.Fill>
        <Binding ElementName="Sdr_Rot" Path="Value" ConverterParameter="1">
            <Binding.Converter>
                <local:DoubleToBrushConverter/>
            </Binding.Converter>
        </Binding>
    </Rectangle.Fill>
</Rectangle>
```

Das Interface `IValueConverter` ist eine Komponente, die die Umwandlung von Werten zwischen verschiedenen Formaten oder Darstellungen ermöglicht. Sie definiert zwei Methoden: `Convert` und `ConvertBack`.

Die `Convert`-Methode wird verwendet, um einen Eingabewert in einen anderen Wert umzuwandeln, der an eine Zielkomponente gebunden werden kann. Der Rückgabewert der `Convert`-Methode ist der konvertierte Wert. Diese Methode wird automatisch aufgerufen, wenn ein Wert von der Quelle zur Zielkomponente gebunden wird.

Die `ConvertBack`-Methode wird verwendet, um einen Zielwert in den ursprünglichen Eingabewert zurückzuwandeln. Diese Methode wird aufgerufen, wenn der Wert von der Zielkomponente zur Quelle zurückgebunden wird. Der Rückgabewert der `ConvertBack`-Methode ist der ursprüngliche Eingabewert.

Mittels `IValueConverter` können eigene Converter-Klassen erstellt werden, welche es ermöglichen situative Umwandlungen mit komplexer Logik selbst zu definieren.

Der zu verändernde Wert findet sich im Parameter `value` wieder. Mittels `targetType` kann der geforderte Zieldatentyp erfragt werden. Anhand `culture` kann mittels eines `CultureInfo`-Objekts die Converter_Logik lokalisiert werden. Mittels `parameter` kann ein weiterer (statischer) Wert von der XAML-Deklaration des Bindungsobjekts mit an den Converter gegeben werden und so dieser für unterschiedliche Situationen definiert werden.

MultiBinding und IMultiValueConverter

- Mittels MultiBinding kann eine Bindung mit mehreren Source-Werten und einem Target etabliert werden.
- Die einzelnen Sources werden in der Bindings-Eigenschaft in Form normaler Binding-Objekte definiert
- Ein obligatorischer MultiValueConverter definiert die Logik, nach der die Source-Werte zu einem Target-Wert kumuliert werden
- IMultiValueConverter funktioniert wie IValueConverter
 - values[] – Array statt value <- Werte in gleicher Reihenfolge, wie Bindings in XAML

```
<MultiBinding>
  <MultiBinding.Converter>
    <local:SourcesToTargetConverter/>
  </MultiBinding.Converter>
  <Binding ElementName="Sdr_Red" Path="Value"/>
  <Binding ElementName="Sdr_Green" Path="Value"/>
</MultiBinding>
```

```
public class SourcesToTargetConverter : IMultiValueConverter{
    public object Convert(object values[], Type targetType, object parameter, CultureInfo culture){...}
```

Soll ein Target-Wert auf mehreren Source-Werten basieren kann ein MultiBinding-Objekt verwendet werden. Dies kann nun mit einzelnen Binding-Objekten befüllt werden, die wiederum die einzelnen Sources definieren.

Ein MultiBinding benötigt zwangsläufig einen Converter für die Logik, nach der die einzelnen Source-Werte zu dem Target-Wert zusammengefasst werden sollen. Diese Converter-Klasse muss das Interface IMultiValueConverter implementieren. Dies funktioniert genauso, wie IValueConverter, hat aber statt des ‚value‘-Parameteres in den Funktionen den Array-Parameter ‚values[]‘. Dieser beinhaltet die Source-Werte in der selben Reihenfolge, wie die Binding-Objekte in der XAML-Datei definiert sind.